

Skaffold + *MERN* Microservices

Complete setup guide for running a MERN microservices stack locally with Kubernetes, hot reload, and automated file sync using Skaffold.

skaffold/v4beta13

Docker Desktop K8s

Node.js + Express

nginx Ingress

Hot Reload

01 Install Skaffold

Get Skaffold running alongside Docker Desktop Kubernetes

KEY CONCEPT

Skaffold watches your files and either syncs changes directly into running containers or triggers a full rebuild — automatically deciding which is faster and appropriate.

STEP 1 – INSTALL

TERMINAL

copy

#!/ macOS

```
brew install skaffold
```

#!/ Windows

```
choco install skaffold
```

#!/ Linux

```
curl -Lo skaffold https://storage.googleapis.com/skaffold/releases/latest/skaffold-linux-amd64.tgz  
chmod +x skaffold && sudo mv skaffold /usr/local/bin
```

VERIFY

TERMINAL

copy

```
skaffold version
kubectl config use-context docker-desktop
kubectl get nodes # should show 1 node Ready
```



Enable Kubernetes first: Docker Desktop → **Settings** → **Kubernetes** → **Enable Kubernetes**.

02 Project Structure

How to organise your microservices for Skaffold

```
mern-app/
├─ skaffold.yaml ← Skaffold config (project root)
├─ core/
│ ├─ dockerfile.dev
│ ├─ .dockerignore ← critical: excludes node_modules
│ ├─ nodemon.json
│ ├─ package.json
│ └─ src/
│   └─ index.js
├─ notification/
│ ├─ dockerfile.dev
│ ├─ .dockerignore
│ ├─ nodemon.json
│ ├─ package.json
│ └─ src/
│   └─ index.js
└─ k8s/
  ├─ core-deployment.yml
  ├─ core-service.yml
  ├─ notification-deployment.yml
  ├─ notification-service.yml
  └─ ingress.yml
```



Each service is its own **build context**. Sync `src` paths are **relative to the service folder**, not the project root — this is the most common config mistake.

03 Dockerfiles & Config

Dev Dockerfiles with layer caching and nodemon for hot reload

LAYER CACHING

Copy `package.json` first, run `npm install`, then copy source code. Docker caches the install layer — when only `.js` files change, the install step is skipped and rebuilds take ~5s instead of ~60s.

dockerfile.dev

CORE/DOCKERFILE.DEV | NOTIFICATION/DOCKERFILE.DEV

copy

```
FROM node:18-alpine
WORKDIR /app

# Cached layer — only re-runs when package.json changes
COPY package*.json ./
RUN npm install

# Source code layer
COPY . .

EXPOSE 3000
CMD ["npx", "nodemon", "src/index.js"]
```

.dockerignore



Without .dockerignore, Scaffold watches `node_modules` too — any change triggers a full rebuild. This is the #1 cause of unnecessary rebuilds.

CORE/.DOCKERIGNORE | NOTIFICATION/.DOCKERIGNORE

```
node_modules
.git
*.log
.env
dist
build
.DS_Store
```

nodemon.json

CORE/NODEMON.JSON | NOTIFICATION/NODEMON.JSON

copy

```
{
  "watch": ["src"],
  "ext": "js,json",
  "ignore": ["node_modules"],
  "delay": 500,
  "legacyWatch": true // required inside containers
}
```

04 Kubernetes Manifests

Deployments, Services and Ingress for both microservices

core-deployment.yml

K8S/CORE-DEPLOYMENT.YML

copy

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: core-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: core
  template:
    metadata:
      labels:
        app: core
    spec:
      containers:
        - name: core
          image: core:latest
          imagePullPolicy: IfNotPresent # use local image, never pull
          ports:
            - containerPort: 3000
          resources:
            limits:
              memory: "128Mi"
```

```
      cpu: "500m"
    requests:
      memory: "64Mi"
      cpu: "250m"
```

core-service.yml

K8S/CORE-SERVICE.YML

copy

```
apiVersion: v1
kind: Service
metadata:
  name: core-service
spec:
  selector:
    app: core
  type: ClusterIP
  ports:
    - name: core-port
      port: 80
      targetPort: 3000
```

ingress.yml



Install nginx Ingress Controller first. See Section O6 for the install command. Check with: `kubect1`

`get pods -n ingress-nginx`.

K8S/INGRESS.YML

copy

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: queue-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - pathType: Prefix
            path: "/"
            backend:
```

```

    service:
      name: core-service
      port:
        number: 80
  - pathType: Prefix
    path: "/api/notification"
    backend:
      service:
        name: notification-service
        port:
          number: 80

```

FIELD	VALUE	WHY IT MATTERS
<code>imagePullPolicy</code>	<code>IfNotPresent</code>	Uses the locally built image — never tries to pull from a remote registry.
<code>ingressClassName</code>	<code>nginx</code>	Selects the nginx controller. Without this, ingress rules are ignored → 404.
<code>rewrite-target</code>	<code>/</code>	Strips path prefix before forwarding. <code>/api/notification/x</code> becomes <code>/x</code> inside the container.
<code>type: ClusterIP</code>	<code>ClusterIP</code>	Services only accessible inside the cluster. External traffic goes through Ingress only.

05 skaffold.yaml

The heart of your setup — build, sync, and deploy config

⚠️ **Cannot mix `infer` and `manual` in the same sync block.** Use only one per artifact. For backend Node services use `manual` — you control the exact destination path explicitly.

SKAFFOLD.YAML

copy

```

apiVersion: skaffold/v4beta13
kind: Config
metadata:
  name: mern-app

# — Build —
build:
  local:

```

```

    push: false          # use local images, don't push to registry
artifacts:

- image: core
  context: core          # build context = ./core folder
  docker:
    dockerfile: dockerfile.dev
  sync:
    manual:              # paths are relative to ./core context
      - src: '**/*.js'    # NOT 'core/**/*.js' ← that would fail
        dest: /app
      - src: '**/*.json'
        dest: /app

- image: notification
  context: notification
  docker:
    dockerfile: dockerfile.dev
  sync:
    manual:
      - src: '**/*.js'
        dest: /app
      - src: '**/*.json'
        dest: /app

# — Manifests —————
manifests:
  rawYaml:
    - k8s/core-deployment.yml
    - k8s/notification-deployment.yml
    - k8s/core-service.yml
    - k8s/notification-service.yml
    - k8s/ingress.yml

# — Port Forwarding —————
portForward:
- resourceType: service
  resourceName: core-service
  port: 80
  localPort: 3000
- resourceType: service
  resourceName: notification-service
  port: 80
  localPort: 4000

```

The `src` path in manual sync is **relative to the build context**. Since `context: core`, Skaffold is already inside the `core/` folder. Writing `core/**/*.js` would look for `core/core/file.js` — which doesn't exist — so every save triggers a full rebuild instead of a fast sync.

06 Running Skaffold

Start the dev loop and verify everything is working

STEP 1 — INSTALL NGINX INGRESS CONTROLLER

TERMINAL

copy

```
kubectl apply -f \
  https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.0/deploy/

kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=90s
```

STEP 2 — START SKAFFOLD

TERMINAL

copy

```
kubectl config use-context docker-desktop

skaffold dev                # standard dev loop
skaffold dev --verbosity=info # see sync vs rebuild decisions
skaffold dev --tail          # stream all pod logs
```

VERIFY

TERMINAL

copy

```
kubectl get pods      # all pods Running
kubectl get services   # core-service, notification-service
kubectl get ingress    # queue-ingress with ADDRESS
```


Service	Via Ingress	Via Port-Forward
Core API	http://localhost/	http://localhost:3000 ✓
Notification API	http://localhost/api/notification	http://localhost:4000 ✓

07 Sync vs Rebuild

When Skaffold syncs files instantly vs triggers a full image rebuild

Save `core/src/routes/user.js` ↓ Matches sync rule: `**/*.js` → /app ↓ File copied directly into running pod ↓ nodemon detects change → server restarts 🔥 ↓ ~1 second total. No rebuild.

`npm install express-validator` ↓ `package.json` updated ↓ `node_modules` changed – not in sync rules ↓ Skaffold triggers full image rebuild ↓ Docker layer cache: only npm install layer reruns ↓ ~5-10s with caching ✓

FILE CHANGED	ACTION	SPEED
<code>src/**/*.js</code>	Synced directly into pod. nodemon restarts.	~1s 🔥
<code>src/**/*.json</code>	Synced directly into pod.	~1s 🔥
<code>package.json</code>	Full rebuild. Docker uses cached npm install layer.	~5-10s
<code>dockerfile.dev</code>	Full rebuild, no layer cache.	~60s

DEBUG SYNC

TERMINAL copy

```
skaffold dev --verbosity=debug 2>&1 | grep -E "sync|rebuild|build"

# Good – sync working:
# Syncing 1 files for core:latest ✓
```

```
# Bad – unnecessary rebuild:  
# Building [core]... ← file wasn't matched by sync rule
```

08 Installing New Packages

Correct workflow for adding npm dependencies while Skaffold is running

KEY CONCEPT

When you run `npm install`, `node_modules` changes locally — but it's excluded from sync rules. Skaffold detects the `package.json` change and triggers a full image rebuild so the package gets installed *inside the container*.

- **Run npm install in the service folder**

```
cd core && npm install express-validator
```

- **package.json updates automatically**

Skaffold is watching and detects the change immediately

- **Skaffold triggers a full rebuild**

Docker layer cache: only the npm install layer reruns (~5–10s)

- **New package available in container** ✓

Pod restarts automatically with updated node_modules

💡 If Skaffold doesn't auto-detect: `Ctrl+C` then `skaffold dev`. Or in another terminal: `skaffold build`.

09 Common Issues & Debugging

Quick fixes for the most frequent Skaffold and Kubernetes problems

ERROR	CAUSE	FIX
<code>ImagePullBackOff</code>	K8s trying to pull from remote registry	Set <code>imagePullPolicy: IfNotPresent</code> in deployment
<code>404 nginx</code>	Ingress class missing or wrong port	Add <code>ingressClassName: nginx</code> , verify service ports match

ERROR	CAUSE	FIX
Rebuilds every save	Wrong src pattern or missing .dockerignore	Use <code>**/*.js</code> (not <code>core/**/*.js</code>), add .dockerignore to each service
sync + infer error	Both sync types on same artifact	Use only <code>manual</code> OR <code>infer</code> per artifact — never both

Useful Commands

TERMINAL

copy

```

skaffold dev                # start dev loop
skaffold dev --tail          # dev loop + stream all pod logs
skaffold dev --verbosity=info # see sync vs rebuild decisions
skaffold build              # rebuild images only, no deploy
skaffold delete            # tear down all K8s resources
skaffold render             # preview final K8s YAML output

# Pod debugging
kubectl logs -f deploy/core-deployment
kubectl exec -it deploy/core-deployment -- /bin/sh
kubectl describe pod <pod-name>

```